01_architect

Principal Architect Review

Headline kill. Helios is a 5-MCP-server / 7-agent / FSM / vector-store / memory-store apparatus erected to answer a question — "did this rate move because behavior changed or because traffic mix changed?" — that the system's own `stats-mcp.decompose_change` answers in **20 lines of arithmetic** (MCP_ARCHITECTURE.md §9, lines 274–288) over a single daily fact table (`fct_daily_funnel`) that has **exactly three usable dimensions** (`device_category`, `country`, `channel_key`; Bible §8.3). Three Opus agents, a best-first hypothesis tree, an adversarial Critic re-query loop, BigQuery-plus-vector memory, and a deterministic FSM are wrapped around a `GROUP BY` and a subtraction. The entire agentic edifice is a delivery mechanism for a query a competent analyst writes — and a notebook runs — in minutes. Nothing here is built; the complexity is all committed, none of the value is.

[CRITICAL] The agentic apparatus is grossly disproportionate to the problem it solves

Claim attacked. "seven agents in a plan-execute-critique loop" with three on Opus (AGENT_ARCHITECTURE.md §3), five MCP servers (MCP_ARCHITECTURE.md §1), a memory store + vector store (Bible §22; DEPENDENCY_MAP T7), and a deterministic FSM (AGENT_ARCHITECTURE.md §1) — all to "diagnose WHY an e-commerce funnel moved." **Why it fails.** The thesis centerpiece is the mix/rate/interaction identity, and the reference implementation of it is `decompose_change`: 14 lines of Python (MCP_ARCHITECTURE.md §9). The grain it runs over, `fct_daily_funnel`, is one row per `(date, channel, device_category, country, is_new_user)` (Bible §8.3) — a table small enough to pull into pandas. The honest scope of the diagnosis is: for each canonical metric, run the decomposition across ≤3 dimensions, rank by rate-effect, attach a z-test and a dollar figure. That is a single notebook cell and one `stats` call. Wrapping it in Orchestrator→Monitor→Decompose→Diagnose→Critic→Prescribe→Narrator, each a separate LLM with its own system prompt, allow-list, retry policy, and typed envelope (AGENT_ARCHITECTURE.md §6), is architecture cosplay. The agent count is justified nowhere by problem difficulty — only by the desire to *look* like a multi-agent system. **Fundamental.** The problem is too small for the machine.

[CRITICAL] "Deterministic finite state machine" is a label laundering stochastic LLMs as deterministic

Claim attacked. "Helios is **not** an autonomous free-roaming agent. It is a **deterministic finite state machine** … transitions are auditable and reproducible" (AGENT_ARCHITECTURE.md §1). The reliability section then concedes "LLM nodes are run at `temperature 0` but are **not** byte-deterministic" (§11). **Why it fails.** The FSM determinism covers only the edges — PLAN→MONITOR→DECOMPOSE — which are trivial and never the source of error. Every decision that *matters* lives inside a node and is made by a sampling LLM: which slice Diagnose drills (§6.4 "drill the slice with the largest rate-effect first" — chosen by the model), which of four refutations the Critic pursues and whether it returns PASS/DOWNGRADE/DROP (§6.5), which story the Narrator tells (§6.7). Calling the wrapper "deterministic" while the wrapped reasoning is non-reproducible (NFR-D2 demands "the same findings set" from a pinned run — unverifiable and likely false for an Opus tree search) is the central architectural sleight of hand. "Reproducible transitions" over irreproducible content is reproducibility theater. **Fundamental.** Determinism is asserted of the shell, not the brain.

[CRITICAL] The wide-fact assumption is already broken: the dimensions the agents are told to drill do not exist on the fact

Claim attacked. `list_dimensions()` "MUST return exactly the canonical dimensions" — 16 of them including `operating_system`, `browser`, `region`, `source`, `medium`, `campaign`, `landing_page`, `item_category`, `item_name` (FR-A7, Bible §6.A); Diagnose expands "along the next canonical dimension" to `MAX_DEPTH=3` (AGENT_ARCHITECTURE.md §6.4, §9). **Why it fails.** `fct_daily_funnel` carries only `channel_key`, `device_category`, `country`, `is_new_user` (Bible §8.3, verified on the artifact). `fct_funnel` adds nothing beyond `device_category` / `country` / `channel_key`. So 11 of the 16 promised dimensions have **no column on the fact the Monitor/Decompose/eval injector all read** (DEPENDENCY_MAP A4.7: "Primary feed for Monitor + Decompose + eval injector"). A best-first tree told to drill 3 dimensions deep will exhaust the real dimensionality at depth ≈2 and then either fail on `UnknownDimension` or silently never explore `landing_page` / `source` — the exact slices where real funnel root causes live. The architecture's headline capability (deep dimensional RCA) is contradicted by its own data model. **Fundamental.** The fact is narrow; the agent contract assumes it is wide.

[HIGH] The guardrail chain is a deep coupled pipeline whose every link is a single point of failure

Claim attacked. "Every governed datapoint flows through this fixed sequence" — `build_query → dry_run → run_query → reconcile → stats → Critic → render_brief` enforced by tool preconditions I1–I4 (MCP_ARCHITECTURE.md §2). **Why it fails.** This is a 7-stage synchronous chain where each stage can hard-fail the datapoint: `NotDryRunFirst` if hashing normalization drifts (the `sha256` gate normalizes whitespace+case, §9 line 246 — any agent that pretty-prints

SQL differently between dry_run and run_query silently breaks I1); `ByteBudgetExceeded` forces the agent to "narrow scope" mid-tree (a control-flow side effect that mutates the analysis); `reconcile ≤0.5%` (G4) can fail a perfectly correct finding on floating-point/rounding drift; `UnknownDimension` is "a hard stop for that branch" (§11). Each guardrail is individually reasonable; chained synchronously inside an LLM tool loop with ≤3 retries and a 5-minute wall-clock, they compound into a brittle pipeline whose failure modes (G3 hash miss, G4 drift, budget mid-run rescope) are all *silent quality degraders*, not loud errors. The operational burden of debugging "why did finding F-2021-0042 get dropped" across 7 servers and 7 agents is enormous for the one person who must run this. **Fixable.** Collapse stages; but coupling is designed-in.

[HIGH] The Critic→Diagnose re-query loop multiplied by the hypothesis tree is an unbounded-in-practice cost/latency bomb

Claim attacked. "<5 min/run" and "≤5 GiB/run" (Bible §5.2, NFR-P1/C1) with `MAX_BRANCHING=4`, `MAX_DEPTH=3`, `MAX_REFUTE_ROUNDS=2` (AGENT_ARCHITECTURE.md §9), three Opus agents, and a Critic that "can re-query to refute" (MCP_ARCHITECTURE.md §10). **Why it fails.** Best-first over 4 branches × 3 depth is up to $4^3 = 64$ nodes, each doing `build_query→dry_run→run_query→significance_test→decompose_change→reconcile` (§6.4) — that is hundreds of MCP round-trips and Opus turns. Every candidate that survives goes to the Critic (Opus), which itself runs `run_query` confound probes and `significance_test` on holdouts (§6.5), and on DOWNGRADE bounces **back to Diagnose for a targeted re-query** up to 2× (§5.1 transition table). The compounding is `nodes × candidates × (1 + refute_rounds)` Opus invocations plus their tool fan-out. Asserting this finishes in <5 minutes and under 5 GiB with three Opus agents and a tree search is a target with no measurement behind it (DEPENDENCY_MAP: "The only artifact that currently exists is the Bible … everything else is remaining"). The sample results table (Bible §20.9, "4m12s/run, 2.1 GB") is a fabricated illustration, not a measurement — there is no runner to produce it. **Fixable.** Caps exist, but the budget is unvalidated and optimistic.

[HIGH] Build-vs-buy: the off-the-shelf stack does 80% of this with none of the maintenance surface

Claim attacked. "Anti-product stance … not a BI dashboard, not an ad-hoc SQL chatbot" (Bible §4.4) — positioned as if no tool covers governed-metric funnel diagnosis. **Why it fails.** A governed semantic layer (the actual keystone, A5.1) + scheduled anomaly detection + funnel decomposition is largely a solved commodity: Lightdash/Cube/dbt-MetricFlow give you the governed metric layer and SQL generation; Amplitude/Mixpanel/GA4 itself give funnel + anomaly + segment-contribution analysis out of the box; a 200-line scheduled notebook does the mix/rate decomposition and emails a brief. Bible §14.6 even admits a "Mapping to dbt semantic layer /

MetricFlow" exists. The *only* genuinely custom asset is `decompose_change` (commodity algebra) and the eval harness. Helios chooses to hand-build a 5-server MCP fabric, a 7-agent SDK orchestration, a bespoke memory+vector store, and a CI eval — for a single static 3-month public dataset. The build cost vastly exceeds the buy-or-notebook cost, and the differentiator (Simpson's-paradox-aware decomposition) is a feature, not a platform. **Fundamental.** The defensible core is one function; the rest is reinvented infra.

[HIGH] One-person maintenance surface is unsustainable and self-contradicted by the "production / multi-tenant" roadmap

Claim attacked. Solo portfolio cadence (DEVELOPMENT_PLAN.md §5: "solo portfolio") maintaining dbt project + 16-dim semantic registry + 5 MCP servers + 7 agents + memory DDL (8 tables, Bible §22) + vector store + eval harness (injector + 50 scenarios + 6 scorers + gates) + CI — while the roadmap promises "Multi-tenant, real-time streaming, warehouse-agnostic" with "Snowflake/Databricks/DuckDB adapters" (Bible §23.4). **Why it fails.** The surface area is enormous: every component has its own contract, error taxonomy (14 error codes, MCP §5), config, and test suite, and they are tightly coupled (a registry column rename must stay in lockstep across the YAML, the marts, the resolver, the Finding envelope, and the eval labels — DEPENDENCY_MAP §8 "Continuity guardrails" is an admission of how fragile the coupling is). The roadmap then layers tenant isolation, streaming ingestion, and three warehouse adapters (§23.4) on top — for a dataset that is **static, historical, obfuscated, and 3 months long** (Bible §23.7). Promising warehouse-agnostic multi-tenancy when **zero lines run** (DEVELOPMENT_PLAN.md §2 "Code layer — not started") is roadmap fantasy that inflates the maintenance commitment without any evidence the single-tenant single-dataset core works. **Fundamental.** Scope outruns the maintainer and the data.

[HIGH] "Always-on / scheduled / autonomous" is architecturally incoherent against a frozen dataset

Claim attacked. "The product's heartbeat is the **autonomous scheduled run**" (CLAUDE.md §1); a scheduler entrypoint (A9.2) "fires" the Orchestrator on a cron (AGENT_ARCHITECTURE.md §10); Monitor does "anomaly detection over time" with `forecast` and seasonality. **Why it fails.** The dataset is "a fixed historical export (~2020-11-01 to 2021-01-31)" (Bible §20.1). Every scheduled run sees byte-identical data; nothing is "fresh"; there is nothing to monitor. The architecture commits a whole scheduling tier (DEPENDENCY_MAP A9.2, M11), a freshness SLA (Bible §15.5, FR-A8 incremental "only new date shards"), a suppression list that decays over runs (PRIOR_HALF_LIFE_DAYS=60), and memory "loop closure" where "a later run sees seen-before" (AGENT_ARCHITECTURE.md §8) — all premised on a stream of new data that does not and will never exist here. The entire "proactive not reactive" pillar (CLAUDE.md §3.5) is unfalsifiable on the

only data the system has. A scheduler firing the same FSM at the same frozen table is a cron job that recomputes a constant. **Fundamental.** "Always-on" has nothing to be on.

[MEDIUM] The "single source of truth" cannot agree on its own filename, schema keys, or scenario count

Claim attacked. "metric/dimension definitions live **only** in `models/semantic/semantic_models.yml`" (CLAUDE.md §5; MCP_ARCHITECTURE.md §3 config points `registry:` at `semantic_models.yml`); the resolver reads `m["sql"]`, `m["name"]`, `m["agg"]` (MCP_ARCHITECTURE.md §9, lines 292–313); "ships **34 scenarios**" (Bible §20.3). **Why it fails.** The file on disk is `models/semantic/semantic_layer.yaml` — wrong name in CLAUDE.md and MCP_ARCHITECTURE.md, so `semantic-mcp` as configured would fail to load its keystone. Its entries use `metric_name:` and `sql_definition:` (verified in the artifact), but `build_query` indexes `m["name"]` / `m["sql"]` / `m["agg"]` and the `Finding` envelope is built around `name` / `sql` — so the resolver cannot read the registry without an adapter that **does not exist** (the file's own header note admits "build_query maps metric_name->name, sql_definition->sql" — an unwritten translation layer). And `eval/scenarios/` contains **50** scenarios across 7 files, not the 34 cited everywhere (Bible §20.3, DEPENDENCY_MAP A10.3, DEVELOPMENT_PLAN.md §8/§12). The governance keystone — the thing the whole anti-hallucination guarantee rests on — has a filename drift, a schema-key mismatch requiring phantom glue, and a count that contradicts the spec. **Fixable.** Rename, write the adapter, recount — but it betrays the discipline.

[MEDIUM] "0 hallucinated columns" is a contract about the keystone, and the keystone is precisely where the artifacts already disagree

Claim attacked. "hallucinated columns are structurally impossible because the model can only reference governed metrics" → "0 hallucinated columns" (Bible §24.2, §4.5, NFR-T1). **Why it fails.** The guarantee is only as strong as the registry-resolver binding, and that binding is currently broken (prior finding): a resolver that can't read `metric_name` / `sql_definition` either crashes or requires hand-glue that reintroduces exactly the manual SQL the architecture swears it eliminates. Worse, the guarantee is about *columns*, not *reasoning*: governed SQL prevents naming a column that doesn't exist; it does nothing to stop Diagnose composing correct columns into a wrong causal story (a deploy, a price change, a competitor, a holiday — none observable in this data). The architecture markets structural impossibility of one narrow failure (bad column names) as if it were trustworthiness of the diagnosis. "0 hallucinated columns" is true and nearly worthless; the failure mode that matters — hallucinated *causation over correct numbers* — has no structural guard at all. **Fundamental.** The guarantee guards the cheap failure, not the expensive one.

[MEDIUM] The Critic is a stochastic LLM grading another stochastic LLM with no ground truth — "adversarial verification" is not a guarantee

Claim attacked. "an adversarial Critic agent tries to refute every finding before it ships" as the third defensibility pillar (Bible §4.5); "Critic (Opus): adversarial reasoning that must out-think Diagnose" (AGENT_ARCHITECTURE.md §3). **Why it fails.** Critic and Diagnose are the *same model family* (both Opus) with correlated blind spots; a confound Diagnose misses, Critic is statistically likely to miss too (shared training, shared priors). The Critic has no oracle — its `recall_prior`/seasonality checks are only as good as a hand-seeded `seasonality_calendar` (DEPENDENCY_MAP §8 warns an "unseeded seasonality_calendar makes the Critic blind"), and its verdicts (PASS/DOWNGRADE/DROP) are sampled tokens, not proofs. Architecturally this adds an entire Opus stage, a bounded re-query loop, and a suppression-list side-channel to obtain a *probabilistic* second opinion that the architecture then presents as the trust pillar. Stacking two samplers does not manufacture a guarantee; it manufactures cost and the *appearance* of rigor while leaving the actual correctness ungrounded (it rests entirely on the offline eval — which grades arithmetic the system itself defines as truth). **Fundamental.** Verification by a peer sampler is not verification.